GrowthPulse LoRaWAN Setup: Web App Checklist (Bryan)

Your side is what makes a teammate's or customer's LoRaWAN node show up in the app automatically. It is already built and deployed on `main` (live on growthpulsecloud.com), so this is the configuration to keep correct, not a fresh build. There is **nothing to collect from a teammate** anymore: no device ID, no shared token. The app auto-provisions each node and routes its data by itself.

What's deployed

- `netlify/functions/provision-lorawan.js` generates OTAA keys, creates the node's TTS device, stores the route in Supabase, and pushes the keys to the board.
- `netlify/functions/register-gateway.js` registers a teammate or customer gateway on TTS when they add it in the app.
- `netlify/functions/lorawan-uplink.js` is the webhook TTS calls; it decodes the 9 raw bytes and forwards the reading into Losant (same pipeline as Wi-Fi).
- `netlify/functions/lorawan-switch-wifi.js` switches a node back to Wi-Fi.
- `firmware/GP_Combined/GP_Combined.ino` is the one firmware that runs both transports.

1. Route table (one-time)

Supabase -> SQL Editor -> run `docs/growthpulse-lorawan-routes-schema-v2.sql`. This creates the `lorawan_devices` table the provisioner writes and the webhook reads. It is what auto-routes each node, so there is no hand-kept map.

2. Netlify environment variables

Site configuration -> Environment variables. These drive the LoRaWAN side:

Variable	Value
`TTS_API_KEY`	TTS API key with create/read/write on end devices **and** gateways
`TTS_APP_ID`	your TTS application id (`growthpulse`)
`TTS_USER_ID`	your TTS username (owns the auto-registered gateways)
`TTS_CLUSTER`	`nam1.cloud.thethings.network` (default if unset)
`TTS_FREQ_PLAN`	`US_902_928_FSB_2` (default if unset)
`LORAWAN_WEBHOOK_TOKEN`	shared secret in the TTS webhook header, so only TTS can call the uplink function

Already set and still required: `LOSANT_APP_ID`, `LOSANT_COMMAND_TOKEN`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`. Redeploy after changing any env var.

> The old `LORAWAN_DEVICE_MAP` (hand-mapping a TTS device to a Losant device) is no longer needed; routing is automatic via the Supabase table. `LORAWAN_DEFAULT_DEVICE_ID` exists only as a last-resort fallback if no route is found.

3. The TTS application + webhook (one-time)

The uplink webhook is configured on the `growthpulse` TTS application: - **Application -> Integrations -> Webhooks -> Custom webhook**, Base URL `https://growthpulsecloud.com/.netlify/functions/lorawan-uplink`, uplink message path `/`, header `X-Webhook-Token` set to the same value as `LORAWAN_WEBHOOK_TOKEN`. - No per-device payload formatter is required; the function decodes the raw bytes itself.

4. Verify it end to end

With a teammate's node provisioned and uplinking: 1. Netlify -> **Functions -> lorawan-uplink -> logs**: each uplink logs `forwarded: true` (about every 60s in test). 2. The node shows live in the app with a **LoRaWAN** badge. 3. Log reads: `401 Bad webhook token` means `LORAWAN_WEBHOOK_TOKEN` does not match the TTS header. `404 No Losant mapping` means the Supabase route was not stored for that TTS device (check that `provision-lorawan` ran and the `lorawan_devices` row exists).

You're done when: the function logs show forwarded uplinks and the LoRaWAN node appears live in the app, with no manual mapping or token exchange.